

Big Data Storage (WDS224 · SPA)

Quelle: WDS224 - Big Data (SPA).zip — 27 Unterrichtseinheiten (UE01–UE27), je Foliensatz + Reader, gegliedert in 6 Blöcke (A–F), plus offizielle Beispielfragen mit Musterlösungen.

Prüfungsformat laut Beispielklausur: 6 Aufgaben / 25 Punkte. Operatoren: *einordnen, vergleichen, begründen, erklären, analysieren*.

0. Die zentrale Denkfigur des ganzen Kurses

Der Kurs hat **eine** durchgehende Argumentationskette. Wer sie beherrscht, kann fast jede Klausurfrage beantworten:

Datenform → Zugriffsmuster → physische Konsequenz → Speicherlogik → (erst dann) Produkt

Merksätze für die Klausur:

- **Rolle vor Produkt, Kriterium vor Wertung, Passung vor Schlagwort.**
- Nie mit einem Produktnamen anfangen. Prüffrage: *Würde meine Begründung noch tragen, wenn ich den Produktnamen streiche?* Wenn nein → Begründung fehlt.
- Ein Speichersystem ist nie „gut“ oder „schlecht“, sondern nur **passend oder unpassend für eine Problemklasse**.

Der Big-Data-Stack als Entscheidungskette

Storage → Syntax/Encoding → Datenmodell → Verarbeitung → Serving

Jede Ebene **begrenzt** die Möglichkeiten der darüberliegenden. Wer an der falschen Ebene optimiert, verschiebt Probleme nur.

BLOCK A — Grundlagen & Trade-offs

UE01 · Kursüberblick und Big-Data-Storage-Perspektive

Leitfrage: Warum ist Storage der Engpass und Strukturgeber für Big Data?

Kernaussagen

- Big Data ist im Kern ein **Skalierungsproblem**, das sich zuerst im Speichern/Verschieben/Lesen zeigt — nicht in Algorithmen.
- Storage ist **nicht neutral**: er erzwingt Trade-offs und macht implizite Annahmen sichtbar.
- Wichtig: Unterschied **Latenz** (Zeit pro Einzelzugriff) vs. **Durchsatz** (Volumen insgesamt).

Workload-Typen und ihre Speicherfolgen

Typ	Ziel	Speicherfolge
OLTP (Online Transaction Processing)	Transaktionen, Korrektheit	starke Konsistenz, schnelle Writes, Indizes
OLAP (Online Analytical Processing)	Analysen, Aggregation	effiziente Scans, denormalisierte Modelle
Batch	Durchsatz, Kosten	sequenzieller Zugriff, grobe Partitionen
Streaming	Latenz, Kontinuität	stabile Ingestion, Event-Modelle

Heuristiken zur Ersteinordnung

Dominanter Zugriff	Konsequenz
Point Lookup	strukturierte Schlüssel, schnelle Indizes
Range Scan	Daten sortiert oder partitioniert vorhalten
Append	sequenziell optimierter Speicher

Zweite Heuristik — Datenlebenszyklus: Was ist *heiß*, was ist *kalt*? Was kostet ein Zugriff in zwei Jahren? Storage ist auch eine Frage der Zeit.

Typische Denkfehler

1. „**Big Data bedeutet nur viel Daten.**“ → Herausforderung liegt oft in *Vielfalt* und *Zugriffsdynamik*. 100 GB können ein System überfordern, wenn Zugriffe chaotisch sind.
2. „**Die Speicherung ist austauschbar.**“ → Speicher bestimmt Kosten, Latenz, Zuverlässigkeit. Speicherwechsel = Architekturwechsel.
3. „**JSON ist eine Datenbank.**“ → JSON ist eine *Syntax*. Ohne Datenmodell und Zugriffsmuster ist keine Speicherentscheidung begründbar.

Vertiefung

Object Stores haben andere Kostenmodelle als lokale Dateisysteme. **Datenbewegung ist oft teurer als Berechnung.** In modernen Data-Lake-Architekturen ist Storage der stabile, günstige Kern; Compute wird elastisch zugeschaltet.

UE02 · Daten, Workloads und Zugriffsmuster

Leitfrage: Wie führt der Weg von der Datenform über das Zugriffsmuster zur passenden Speicherlogik?

Datenformen

Form	Merkmal	Beispiele
Tabellarisch / strukturiert	stabiles Schema, klare Felder/Typen, Schlüssel & Integritätsbedingungen	Bestellungen, Rechnungen, Stammdaten
Semi-strukturiert	gemeinsame Grundidee, aber nicht jedes Feld in jedem Satz	JSON-Dokumente, Event-Logs
Unstrukturiert	große Objekte; Fachinformation steckt oft in Metadaten	Bilder, Audio, Video, Archive

 **Die Datenform grenzt den Suchraum ein, entscheidet aber nicht final.** Millionen Bestellungen, die nur noch periodisch analysiert werden, führen zu Scan-Logik statt Transaktionslogik. Ein JSON-Dokument mit schnellen Einzelzugriffen wird indexgestützt organisiert.

Zugriffsmuster → physische Konsequenz → Speicherlogik

Zugriffsmuster	Typische Frage	Physische Konsequenz	Naheliegende Speicherlogik
Point Lookup	Welcher Datensatz gehört zu genau diesem Schlüssel?	Indizes oder schlüsselnahe Ablage minimieren Suchkosten	transaktionales DBMS, Key-Value-nahe Logik
Range Scan	Welche Datensätze liegen in einem Wertebereich / Zeitfenster?	Ordnung, Sortierung, Partitionierung senken Kosten zusammenhängender Lesebereiche	scanfreundliche Ablage, analytische Speicherformen
Append	Wie können viele neue Ereignisse fortlaufend aufgenommen werden?	Sequentielles Schreiben + spätere Verarbeitung statt ständiger Reorganisation	Log-, DFS- oder Object-Store-nahe Aufnahme
Batch Read	Wie liest man große Datenmengen effizient am Stück?	Durchsatz, Datenlokalität, große Lesevorgänge > Einzellatenz	Distributed File System, Object Store, analytische Dateiformate

OLTP = Bündel aus Point Lookups + kleinen Updates. **OLAP** = scanlastig + aggregationsorientiert. Die Workload-Begriffe sind also keine Etiketten, sondern *typische Bündel von Zugriffsmustern*.

Mini-Case „ShopNow“ — drei Datenarten, drei Speicherlogiken

Datenart	Datenform	Dominantes Muster	Herleitung → Speicherlogik
Orders	tabellarisch, stark strukturiert	Point Lookup + kleine Updates	Struktur + Integrität + Einzelzugriffe ⇒ transaktional, indexgestützt
Events (Klicks, Suchen, Warenkorb)	semi-strukturiert, variable Attribute	Append, später Range Scan / Batch Read	flexibles Schema + hoher Schreibdurchsatz + spätere Bereichsauswertung ⇒ DFS / Object Store / Lake-Logik
Media (Bilder, Videos, PDFs)	unstrukturiert im Objekt, strukturiert in Metadaten	gezieltes Lesen großer Objekte	Objektablage getrennt von Metadatenzugriff ⇒ Object Store + DB für Metadaten

Kernbotschaft: „Ein Unternehmen“ ist kein sinnvolles Speicherkriterium. Die fachliche Einheit einer Anwendung erzeugt **nicht** automatisch eine einheitliche physische Speicherstruktur. Systemwahl passiert auf Ebene eines **konkreten Zugriffspfads**.

Vertiefung: Warum schnelle Hardware schlechte Organisation nicht heilt

Beruhet ein Workload auf Range Scans, helfen schnellere Einzelzugriffe nur bedingt, solange die Daten physisch ungeordnet liegen. Hardware ändert die absolute Leistung, aber **nicht den Grundkonflikt zwischen Zugriffsmuster und physischer Organisation**.

UE03 · Partitionierung und horizontale Skalierung

Leitfrage: Wie verteilt man Daten so, dass Last und Wachstum beherrschbar bleiben?

Saubere Trennung: *Partitionierung* beantwortet, wie **Last und Datenmenge** verteilt werden.

Replikation (UE04) beantwortet, wie **Ausfälle abgedeckt und Lesezugriffe vervielfältigt** werden.

Ziele der Partitionierung

1. **Skalierung** — horizontal statt vertikal: mehrere Knoten statt eines immer größeren.
2. **Parallelität** — mehrere Knoten beantworten gleichzeitig unterschiedliche Teilmengen (Bearbeitungsbreite).
3. **Begrenzung lokaler Kosten** — eine Anfrage, die nur eine Partition berührt, ist billiger. Latenz hängt also auch davon ab, *wie viele Partitionen* eine Anfrage berührt.

Was Partitionierung *nicht* leistet: Sie erzeugt neue Fragen — langfristige Lastbalance, ungleiche Zugriffe, Verschiebung bei Wachstum, Semantik mehrteiliger Operationen.

Range- vs. Hash-Sharding

Kriterium	Range-Sharding	Hash-Sharding
Grundidee	Zusammenhängende Werte bleiben zusammen	Schlüssel werden über Hashfunktion gleichmäßig verteilt
Stärke	gute Bereichsabfragen, fachlich nachvollziehbare Ordnung	meist gleichmäßigere Last bei key-basierten Einzelzugriffen
Risiko	heiße Bereiche erzeugen Hotspots	Bereichsabfragen werden teuer, weil Ordnung verloren geht
Typischer Fit	Zeitfenster, Regionen, Archive, Bereichsscans	Nutzerprofile, Key-Value-Workloads, viele Point Lookups

Hotspots — das zentrale Praxisproblem

Ein Hotspot entsteht, wenn **Last nicht proportional zur Datenmenge verteilt** ist.

- **Heißer Schlüssel:** extrem viele Zugriffe auf genau einen Wert / ein kleines Cluster.
- **Heißer Bereich:** aktuelle Zeitfenster, trendende Regionen, beliebte Kategorien. → **Range-Sharding ist hierfür besonders anfällig**, weil die fachliche Ordnung gerade die stärker nachgefragten Werte zusammenhält.

Gegenmaßnahmen (keine ist gratis!):

1. Partitionen **feiner schneiden** → heiße Bereiche werden kleiner.
2. **Key-Design anpassen** — Salting, technische Präfixe.
3. **Rebalancing** — gezielte Neuverteilung bestehender Partitionen.

Rebalancing & Consistent Hashing

- Bei **einfachem Hashing** geht die Knotenanzahl direkt in die Zuordnung ein → ein neuer Knoten ändert einen *großen Teil* aller Zuordnungen → viel Datenbewegung.
- **Consistent Hashing:** Schlüssel *und* Knoten werden auf einen **Ring** abgebildet. Ein Schlüssel gehört dem nächsten Knoten im Uhrzeigersinn. Beim Hinzufügen/Entfernen eines Knotens sind **nur benachbarte Ringabschnitte** betroffen → lokales, günstiges Rebalancing.
- **Virtuelle Nodes:** Ein physischer Knoten erscheint **mehrfach** im Ring und übernimmt viele kleine Teilbereiche statt weniger großer → feinere, robustere Lastverteilung.



Allgemeines Prinzip hinter virtuellen Nodes: Verteilung wird robuster, wenn wenige große Einheiten in viele kleine aufgeteilt werden (gilt auch für Task-Scheduling, Datenblöcke etc.).

Klausur-Reihenfolge: erst Zugriffsmuster bestimmen → dann Lastbild benennen → dann Strategie begründen → dann typisches Risiko nennen.

UE04 · Replikation, Konsistenz und CAP

Leitfrage: Wie balancieren verteilte Systeme Konsistenz und Verfügbarkeit?

Replikation ≠ Backup. Backup dient der Wiederherstellung *im Nachhinein*. Replikation dient der fortlaufenden Betriebsfähigkeit während des Problems.

Ziele der Replikation

1. **Ausfallsicherheit / Redundanz** — eine Kopie fällt aus, eine andere übernimmt.
2. **Leseentlastung und regionale Nähe** — Reads auf mehrere Knoten/Regionen verteilen.
3. **Preis:** Mehr Kopien = mehr Abstimmungsaufwand. Je strenger die Konsistenzanforderung, desto früher müssen sich die Kopien einigen → **höhere Latenz, mehr Störungsanfälligkeit**.

Konsistenz als Sichtbarkeitsversprechen

Aspekt	Strong Consistency	Eventual Consistency
Sichtbarkeit nach Write	Alle nachfolgenden Reads sehen denselben neuen Stand	Kopien dürfen vorübergehend abweichen
Stärke	hohe fachliche Eindeutigkeit	hohe Verfügbarkeit, oft geringere Latenz
Preis	mehr Koordination, Wartezeit, störungsempfindlich	stale reads , zeitweise unterschiedliche Sichten
Typischer Fit	Zahlungen, kritische Buchungen, Sitzplatzvergabe, Inventarabbuchung	Produktkataloge, Feeds, global verteilte Inhalte

„Eventual“ heißt *nicht* „irgendwann vielleicht“, sondern **„mit zeitlicher Verzögerung und ohne globale Sofortsicht“**. Ein stale read ist dann kein Fehler, sondern erwartbare Konsequenz des gewählten Trade-offs.

Read-Your-Writes als pragmatische Zwischenform: Nicht jeder Client muss global denselben Stand sehen — es reicht oft, wenn *derselbe Nutzer seine eigene Änderung* unmittelbar wiederfindet. → **Konsistenz ist kein Alles-oder-Nichts-Schalter**.

CAP — was es wirklich sagt

- CAP beschreibt, **was bei Netzpartitionen passiert** — nicht drei frei wählbare Eigenschaften in jeder Lage.
- **Partition** = Teile des Systems können nicht mehr sicher miteinander kommunizieren.
- **Im Störfall ist P gesetzt** (keine Wahl, sondern Störungslage). Die echte Entscheidung ist dann **C oder A**:
 - **C priorisiert:** Anfragen werden abgewiesen oder blockiert, um Konsistenz zu schützen.
 - **A priorisiert:** System bleibt verfügbar, akzeptiert vorübergehend inkonsistente Sichtstände.

CAP an Szenarien:

Szenario	Priorisierung	Warum
Zahlungsdienst	C	Zwei Regionen dürfen denselben Zahlungsvorgang nicht unterschiedlich bewerten
Globaler Produktkatalog	A	kurzzeitig alter Preis/Lagerstand tolerabel, Dienst muss erreichbar bleiben
Social Feed	A (eventual)	geringe Sichtbarkeitsverzögerung < vollständige Nichtverfügbarkeit

! ACID vs. CAP sauber trennen

- **ACID** = Rahmen für **lokale Transaktionen**: Atomarität, Konsistenz (im DB-Sinn), Isolation, Dauerhaftigkeit. → *Wie läuft eine einzelne Transaktion korrekt ab?*
- **CAP** = Unmöglichkeit, bei **Netzpartitionen** gleichzeitig volle Konsistenz und volle Verfügbarkeit zu garantieren. → *Welche Priorität setzt ein verteiltes System unter Kommunikationsstörung?*
- **✗ „ACID widerspricht CAP“** vermischt lokale Transaktionsgarantien mit verteilten Störungssituationen.

Klausur-Reihenfolge: erst fachliche Kritikalität benennen → dann akzeptables Sichtbarkeitsniveau → erst daraus die Priorisierung zwischen C und A.

BLOCK B — Paradigmen, Modelle, Auswahl

UE05 · Storage-Paradigmen im Überblick

Leitfrage: Welche grundlegenden Speicherparadigmen gibt es und wofür eignen sie sich?

Ein Paradigma beantwortet drei Grundfragen: **(1) Wie wird adressiert?** **(2) Welche Struktur wird mitgeliefert?** **(3) Welche Zugriffe liegen nahe?** Ein Paradigma ist **weder** eine Produktklasse **noch** eine einzelne Software.

Paradigma	Adressierung	Typische Stärke	Typische Grenze
Block	Offset / Blockadresse	Basis für Betriebssysteme, VMs, Engines; flexibel & performant	keine Fachstruktur, keine Dateisicht
File	Pfad und Verzeichnis	vertraute Ordnungslogik, intuitiv für Zusammenarbeit	Namensraum & Metadaten werden skalierungsrelevant
Object	Objekt-ID + Metadaten	große BLOBs, Archive, webnahe Nutzung, Data Lakes	kein klassischer Dateibaum, keine reichhaltige Query-Semantik
Key-Value	Schlüssel	schnelle Direktzugriffe, einfache Zugriffslogik	eingeschränkte Abfragesprache
DB-nah	Modell und Query	strukturierte Fachdaten, gezielte Abfragen	je nach Modell komplexer/teurer im Betrieb

Die drei Ordnungslogiken im Kontrast

File	→ /projekt/report.pdf	(Pfad: Platz im hierarchischen Namensraum)
Object	→ obj-847 + Metadaten	(Objekt-ID: Einheit ohne Verzeichnisbaum)
DB	→ SELECT ... WHERE ...	(Modell+Query: Teil einer strukturierten Welt)

⊘ Typische Fehlentscheidungen

- Jede große Medienmenge in ein klassisches Dateisystem pressen → ignoriert Stärken objektartiger Verwaltung.
- Jedes Fachproblem reflexhaft in einer Datenbank lösen → unterschätzt Anforderungen großer Archive/BLOB-Welten.
- Object Store als 1:1-Ersatz eines Dateisystems lesen → Dateisicht ist auch eine Frage der *erwarteten Operationen*.

Antwortreihenfolge: Zuerst *wie wird zugegriffen?* → dann *welche Struktur sollen die Daten mitbringen?* → erst am Ende: welches Paradigma.

UE06 · Datenmodelle, Schema und Validierung

Leitfrage: Wie beeinflusst das Datenmodell die Art der Abfragen und Speicherung?

Dieselbe Bestellung kann relational (Tabellen + Joins), dokumentorientiert (eingebettete Positionen), als Graph (Kunde–Produkt–Bestellung) oder als Cube-Kennzahl erscheinen. **Dieselbe Fachwelt nimmt je nach Nutzung eine andere Form an.**

Modell	Kernlogik	Typische Stärke	Typischer Preis
Relational	Tabellen und Joins	klare Struktur, Integrität, transaktionsnahe Fachdaten	Join-Kosten, besonders bei verteilten/tiefen Zugriffspfaden
Dokument	zusammengehörige Daten als Einheit	natürlich für Objektwelten und variable Felder	Redundanz, schwierigeres globales Aktualisieren
Graph	Beziehungen als erstklassige Struktur	Pfade, Netzwerke, Nachbarschaften	für reine Aggregation oft ungünstiger
Cube (multidimensional)	Kennzahlen entlang von Dimensionen	analytische Verdichtung und Reporting	kein Ersatz für operative Detailmodelle

Schema & Validierung als Qualitätslogik

- **Schema** beschreibt erwartete Struktur/Felder/Typen/Beziehungen. **Validierung** prüft, ob eine Instanz das einhält.
- Praxisbeispiel: Existieren für denselben Status `paid`, `Payed` und `PAYD`, liegt das Problem **nicht in der Syntax, sondern in fehlender Steuerung**.

	Schema on Write	Schema on Read
Prüfung	beim Schreiben gegen ein erwartetes Modell	flexibel ablegen, Interpretation später
Vorteil	frühe Qualitätskontrolle, einfachere Weiterverarbeitung	gut bei heterogenen Quellen / offener Analyse
Preis	geringere Flexibilität bei Strukturänderungen	Komplexität verschiebt sich in Auswertung, Bereinigung, Konsolidierung

Denormalisierung als bewusster Trade-off

- **Normalisiert:** sauber aufgeteilt, wenig Redundanz, konsistente Änderungen.
- **Denormalisiert:** bewusst zusammenkopiert/eingebettet, damit häufige Zugriffe schneller/verteilter möglich werden.
- Nutzen vor allem bei **read-lastigen Systemen**. Preis: **Redundanz steigt, Änderungen müssen an mehreren Stellen nachgezogen werden**.
- ⇒ Denormalisierung ist **kein Trick, sondern ein Tausch zwischen Lesekosten und Änderungskosten**.

💡 Vertiefung: „NoSQL = schemafrei“ ist falsch

Es verschiebt sich meist nur der **Ort der Durchsetzung**: weniger streng beim Schreiben, dafür Regeln im Anwendungscode, in Validierungsregeln oder in späteren Pipelines. **Weniger starres Schema ≠ weniger Bedarf an Regeln**.

UE07 · Auswahlkriterien und Einordnung von Storage-Systemen

Leitfrage: Wie treffe ich eine begründete Speicherentscheidung?

Schlüsselidee: Speicherentscheidungen sind Gewichtungentscheidungen.

Die fünf Kernkriterien

Kriterium	Worauf es zielt	Typischer Zielkonflikt
Latenz	schnelle Antwort pro Zugriff	starke Koordination/Verteilung erhöhen Wartezeit
Durchsatz	viel Last und Volumen insgesamt	hoher Durchsatz garantiert keine kurze Einzelantwort
Konsistenz	eindeutiger sichtbarer Zustand	kostet oft Latenz oder Verfügbarkeit
Skalierung	Wachstum von Daten und Last mittragen	braucht Verteilung und zusätzliche Komplexität
Kosten	Betrieb bleibt organisatorisch tragbar	billige Einfachheit und hohe Leistung fallen nicht zusammen

⚠️ **Latenz ≠ Durchsatz.** Ein System kann viele Anfragen insgesamt bearbeiten und trotzdem für einzelne Zugriffe zu langsam sein. ⚠️ **Kosten ≠ Hardwarepreise.** Betriebsaufwand, Teamwissen, Fehlersuche, Monitoring, Migrationsfähigkeit, organisatorische Komplexität zählen mit.

Die Entscheidungsmatrix

Ihr Nutzen liegt **nicht** darin, automatisch die Antwort auszuspeucken, sondern **Annahmen explizit zu machen**. Eine Matrix funktioniert nur, wenn ihre Begründungen mit dem Workload verknüpft sind.

Von Kriterium zu Architekturfolge

Kriterium hoch gewichtet	Architekturfolge
Latenz	kürzere Zugriffspfade, voraggregierte Sichten, lokalere Datenhaltung
Durchsatz	batch-orientierte Writes, verteilte Lastaufnahme, robuste Ingestion-Pfade
Konsistenz	bewusste Koordination; bestimmte Vereinfachungen werden unmöglich
Skalierung	Partitionierung, Replikation, verteilte Betriebsmodelle
Kosten	jede Entscheidung unter Betrieb & Teamfähigkeit lesen

Fallbeispiel: Monitoring-Plattform für Filialen

Hunderte Filialen liefern im Minutentakt Metriken; regionale Dashboards warnen bei Störungen; zentrales Team analysiert historische Reihen.

- **Durchsatz: dominant** (laufend neue Messwerte)
- **Skalierung: hoch** (Filialen, Sensoren, Messintervalle wachsen)
- **Latenz: differenziert** — niedrig fürs Echtzeit-Dashboard, höher für tiefe Historien-Auswertungen
- **Konsistenz: differenziert nach Datenart** — nicht jede Metrik braucht Buchungsstrenge
- **Kosten: begrenzende Realität** über Jahre

Typische Begründungsfehler

1. **Symptom mit Ursache verwechseln** — „Dienst langsam ⇒ anderes Produkt“, obwohl das Zugriffsmuster oder die Modellierung schuld ist.
2. **Pauschale Superlative** — „maximal skalierbar“, „maximal konsistent“ lösen den Zielkonflikt nicht.
3. **Produktantwort ohne Herleitung.**

BLOCK C — NoSQL

UE08 · Motivation und Konzept von NoSQL

Leitfrage: Warum entstanden NoSQL-Systeme und was versprechen sie?

NoSQL ist **keine ideologische Abkehr von SQL**, sondern eine Reaktion auf konkrete Probleme. Der Begriff bezeichnet keine Technik, kein einheitliches Konsistenzmodell, kein gemeinsames Schema — sondern eine **Familiensammlung**.

Drei zentrale Treiber

1. **Skalierungsdruck** — Lastspitzen, Verteilungsprobleme, Koordinationskosten bei Millionen gleichzeitiger Nutzer.
2. **Latenz und globale Nähe** — strenge zentrale Koordination bremst bei geografisch verteilten Nutzern.
3. **Flexible Datenformen** — Kataloge, Eventströme, Profile ändern sich schneller, als starre Tabellenstrukturen nachgezogen werden.

Wann relationale Muster unter Druck geraten: *Kombination* aus globaler Verteilung + hohen Lastmengen + aufwendigen Joins + flexiblen Daten. → Folgerung ist **nicht** „relational ist schlecht“, sondern „bestimmte Lastprofile verlangen andere Antworten“.

ACID vs. BASE

BASE = **B**asically **A**vailable, **S**oft state, **E**ventually consistent.

Aspekt	ACID	BASE
Grundidee	lokale Transaktionskorrektheit, klare Eigenschaften von Operationen	verteilte Verfügbarkeit und spätere Angleichung
Sichtbarkeit	eher sofort und eindeutig im gewählten Rahmen	darf zeitversetzt und zwischenzeitlich unterschiedlich sein
Passung	kritische Buchungen, eindeutige Zustandsänderungen	webnahe Last, globale Verteilung, tolerierbare Abweichungen
Preis	starke Koordination kann teuer werden	mehr Komplexität bei Sichtbarkeit und Dateninterpretation

BASE bedeutet nicht „weniger Regeln“, sondern „andere Regeln und andere Prioritäten“.

Die vier Familien

Key-Value → Direktzugriff über Schlüssel
 Document → flexible fachliche Einheiten
 Wide Column → große verteilte Tabellenwelten, last-/verteilungsorientiert
 Graph → Beziehungen und Pfade

Fallbild: Globale Handelsplattform

- **Katalogbereich:** flexible Datenformen + globale Leselast ⇒ Dokumentorientierung / denormalisierende Muster plausibel.
- **Warenkorb / Bezahlprozess:** Eindeutigkeit und klare Zustandswechsel dominieren ⇒ NoSQL ist **nicht** die pauschale Antwort.

Drei Fehlteile

1. „NoSQL ist grundsätzlich schneller.“ → Geschwindigkeit hängt vom Zugriffsmuster und der Architektur ab.
2. „NoSQL ist schemafrei.“ → Viele Systeme gehen nur *flexibler* mit Schema um oder organisieren Validierung anders.
3. „NoSQL löst Konsistenzprobleme, indem es sie ignoriert.“ → Die Frage verschiebt sich dahin, *wie* Konsistenz unter Verteilung interpretiert und umgesetzt wird.

UE09 · Key-Value Stores

Leitfrage: Warum sind Key-Value Stores so skalierbar, und wo liegen ihre Grenzen?

Die API als fachliche Verengung

```
put(k, v)  get(k)  delete(k)
```

Diese API ist **kein technisches Detail, sondern die Logik des Systems**. Die wichtigste Rückfrage bei jedem Use Case: **Ist der zentrale Zugriffspfad tatsächlich schlüsselbasiert?**

Aspekt	Stärke	Grenze
API	wenige, klare Operationen	wenig Ausdruckskraft für komplexe Fragen
Skalierung	Keys lassen sich gut verteilen	Verteilung löst keine Analyseanforderung
Latenz	kurze Zugriffspfade möglich	nur, wenn der Key bekannt ist
Modellierung	einfaches mentales Modell	Beziehungen und Bereiche sind sperrig

Warum Hashing so gut passt

Wenn der Zugriff über den Schlüssel läuft, organisiert man auch die **Verteilung** über den Schlüssel. Nicht erst eine komplexe Query wird auf viele Knoten aufgelöst — **schon der einzelne Key zeigt auf die zuständige Stelle.**

Consistent Hashing (Ringmodell): Ein Key gehört dem nächsten Knoten **im Uhrzeigersinn**. Bei Knotenzugang/-ausfall verschiebt sich nur ein Teilbereich. **Virtuelle Knoten:** physischer Server erscheint mehrfach im Ring → feinere, robustere Lastverteilung.

Konsistenz, Versionierung, Konflikte

 **Fehlvorstellung:** „Key-Value Stores sind automatisch eventual consistent.“ Das Modell sagt zunächst nur, *wie adressiert wird*. Replikation/Konsistenz/Konfliktbehandlung ist eine **zweite Frage**.

- **Last-Write-Wins:** praktisch, aber verliert möglicherweise Information.
- **Versionierung / Vektoruhren:** machen sichtbar, dass mehrere Versionen nebeneinander existieren können. Denkrichtung: *erst fragen welche Versionen entstanden sind, dann entscheiden wie zusammenführen*.

Einsatz & Fälle

✓ **Passend:** Session Stores, Caches, Zustandsinformationen, Profile/Konfigurationswerte — solange der Schlüssel den Zugriff vollständig führt. ✗ **Unpassend:** freie Such-/Beziehungslogik, Filter über viele Attribute, Bereichssuchen, Pfade in Beziehungen.

Mini-Case Nutzerprofile mit zwei Zugriffswelten: Login-Pfad (Direktzugriff über Nutzer-ID) ⇒ Key-Value passt. Empfehlungs-/Analysefragen (Filtern nach Interessen, Verhalten, Regionen) ⇒ Key-Value passt nicht. **Systemwahl erfolgt auf Ebene des Zugriffspfads, nicht des Unternehmens.**

Merksatz: Key-Value Stores sind stark, wenn der Schlüssel bekannt ist; sie sind schwach, wenn die Frage erst noch gefunden werden muss.

UE10 · Document Stores

Leitfrage: Wann sind Dokumente das passende Datenmodell?

Nicht „speichert JSON“, sondern: welche fachliche Einheit soll gespeichert werden? Ein Dokument fasst Informationen so zusammen, dass sie häufig **gemeinsam** gelesen, geschrieben oder transportiert werden.

Gute Dokumentmodelle sind **bewusst strukturiert**: stabiler Kern + variable/eingebettete Teile. Auch Document Stores leben von Konventionen, Validierungen und bewussten Modellierungsentscheidungen.

Die Kernentscheidung: Einbettung vs. Referenz

Frage	Einbettung	Referenz
Gemeinsamer Zugriff	sehr häufig gemeinsam gelesen	eher getrennte Zugriffe
Update-Verhalten	zusammen änderbar	separat gepflegt oder geteilt
Wachstum	begrenzt und kontrollierbar	groß oder dynamisch wachsend
Redundanz	akzeptabel, wenn Zugriffsvorteil hoch ist	eher vermeiden, wenn Konsistenz kritisch ist

Beispiel: Bestellung mit Lieferadresse und Positionsliste → **Einbettung**. Kundenobjekt, das in vielen Bestellungen auftaucht und unabhängig aktualisiert wird → **Referenz**.

Abfragen brauchen trotzdem Indizes

Gerade **weil** Dokumente viele Felder/Varianten haben können, müssen zentrale Query-Pfade bewusst gestaltet werden. Reihenfolge: *welche Anfragen kehren wieder?* → *welche Felder sind relevant?* → *dann Indexentscheidung*.

Grenzen

Viele Beziehungen zwischen separaten Einheiten, Multi-Dokument-Transaktionen, häufige Änderungen an redundant eingebetteten Informationen.

Mini-Case: Produktkatalog vs. Auftragswelt

- **Katalog:** variable Attribute, lesenahe Darstellung ⇒ Dokumentmodell natürlich.
- **Auftragswelt:** konsistente Zustandsänderungen über Kunde, Zahlung, Lager, Statuswechsel ⇒ sorgfältiger prüfen.
- **Lehre:** Weder aus der Katalogstärke auf die ganze Fachwelt schließen, noch das Modell wegen der Auftragswelt komplett ablehnen.

Merksatz: Ein Document Store passt dort, wo fachliche Einheit und Zugriffseinheit eng zusammenfallen.

Starke Antwort: auch die Gegenposition mitdenken — *warum wird ein bestimmter Teil gerade **nicht** eingebettet?*

UE11 · Wide Column Stores

Leitfrage: Wie funktionieren Column Families und wofür sind sie optimiert?

 Der Name führt in die Irre. Es geht **nicht** um „viele Spalten“, sondern darum, dass der Zugriff **über einen dominanten Schlüssel und logisch getrennte Spaltenfamilien** effizient wird.

Die drei Bausteine

```
Row Key: sensor-17#2026-04-16
├─ Family "meta"   : typ | standort
├─ Family "values" : 08:00 | 08:05 | 08:10
└─ Family "flags"  : alarm | wartung
```

- **Row Key** = die zentrale Ordnungsentscheidung: bestimmt Ablage, Auffinden, oft die Partitionierung.
- **Column Family** = Gruppierung von Spalten, die aus Zugriffs-/Organisationssicht zusammengehören.
- **Cell** = konkreter Wert an der Stelle aus Zeile, Family und Spaltenname.

Wide Rows & Sparse Columns

- **Wide Rows:** sehr viele zusammenhängende Werte zu einem Schlüssel (ideal für Zeitreihen).
- **Sparse Columns:** nicht jede Zeile muss alle Felder enthalten. Im relationalen Denken wirkt das wie Unordnung — hier ist es eine **Stärke**.

⚠ Abgrenzung zum analytischen Column Store: Ein *analytischer* Column Store organisiert Werte derselben Spalte spaltenweise für Aggregationen. Ein *Wide Column Store* organisiert primär entlang eines **Schlüssels** und gruppiert Familien für operative/semi-analytische Zugriffspfade. Beide tragen „column“ im Namen, lösen aber **unterschiedliche Probleme**.

Der Row Key ist die eigentliche Modellentscheidung

Zugriff „alle Werte eines Sensors für einen Tag“ verlangt einen anderen Key als „alle Sensoren eines Standorts zu einem Zeitpunkt“.

⊘ Row-Key-Fehlmuster:

1. Fachlich eindeutig, **stützt aber den dominanten Leseweg nicht** (Sensor-ID ohne Zeitbezug bei Zeitraum-Abfragen).
2. Lesenähe, **unterstützt aber die Verteilung über Knoten schlecht** → Last sammelt sich in einzelnen Bereichen.

Write Path & LSM-Idee

LSM-Prinzip: Neue Daten werden zunächst schnell aufgenommen und **später geordnet zusammengeführt**. Der Aufwand verschiebt sich vom unmittelbaren Schreiben in spätere Organisationsschritte → passt gut zu vielen laufenden Writes (Log-, Ereignis-, Zeitreihenwelten).

HBase vs. Cassandra (grobe Einordnung)

Frage	HBase	Cassandra
Grundbild	eher mastergestützte Verwaltung von Regionen	peer-to-peer orientierte Knotenstruktur
Typische Einordnung	eng im Hadoop-/HDFS-Umfeld verankert	stark auf Verteilung und Replikation ausgelegt
Zugriffslogik	Row-Key-orientiert, Family-Konzept klar sichtbar	ebenfalls schlüssel- und verteilungsorientiert
Didaktischer Fokus	Architekturidee und Modellpfad	Replikation, Verteilung, dezentrale Sicht

Abgrenzung zu Dokumenten und Tabellen

- **Dokument fragt:** Was gehört als Einheit zusammen?
- **Wide Column fragt:** Welche Werte sollen entlang welcher Reihenfolge schnell gelesen werden?
- Gegenüber relationalen Tabellen: **Uniformität verliert an Bedeutung**.

Merksatz: *Wide Column Stores passen dort, wo geordnete Schlüsselbereiche und vorbereitete Zugriffspfade wertvoll sind.*

UE12 · Konsistenzmodelle in NoSQL-Systemen

Leitfrage: Wie lassen sich Konsistenz und Verfügbarkeit konkret austarieren?

Die Modelle

Modell	Kernaussage	Typische Einordnung
Strong Consistency	Reads sehen nach dem Write den aktuellen Stand	kritische Fachobjekte: Buchung, Zahlung
Eventual Consistency	Unterschiede dürfen zeitweise bestehen	weniger kritische, hochverteilte Daten
Read-Your-Writes	der eigene Write ist für denselben Nutzer sichtbar	Profil, Warenkorb, Einstellungen
Monotonic Reads	spätere Reads fallen nicht auf ältere Stände zurück	stabile Nutzerwahrnehmung

Quorum: N, R, W — die Trade-offs als Regel

- **N** = Anzahl der Replikate
- **W** = Anzahl der Replikate, die einen **Write** bestätigen müssen
- **R** = Anzahl der Replikate, die für einen **Read** befragt werden

★ **R + W > N**

Diese Regel erzwingt einen **Schnitt (Überlappung)** zwischen dem Set der Write-Bestätigungen und dem Set der Read-Befragungen. Mindestens ein Replikat hat sowohl den Write gesehen als auch Einfluss auf den späteren Read → hohe Chance, einen aktuellen Stand zu lesen.

Rechenbeispiele:

Setting	R+W vs N	Bewertung
N=3, W=2, R=2	4 > 3 ✓	Überlappung erzwungen — Read sieht sehr wahrscheinlich aktuellen Stand
N=5, W=2, R=2	4 < 5 ✗	Keine garantierte Überlappung — Read kann einen Bereich treffen, der den jüngsten Write nicht gesehen hat

Steuerungslogik:

- ↑ **W** → bessere Write-Sicherheit, aber langsamere Writes
- ↑ **R** → höhere Chance auf aktuelle Reads, aber teurere Lesevorgänge

⚠ **R+W>N ≠ „immer sicher“**. Reale Systeme haben Netzwerklatenzen, ausfallende Knoten, veraltete Zeitstempel. Quorum ist eine **Gestaltungsregel mit Kosten**, kein magisches Sicherheitszeichen.

Konfliktlösung und Reparatur

- **Read Repair:** Unterschiede werden **beim Lesen** erkannt und ausgeglichen.
- **Write Repair:** Replikate werden nach einem Write später aktualisiert.
- **Last-Write-Wins:** jüngster Zeitstempel entscheidet — **einfach ≠ fachlich gut**, kann zu stillen Informationsverlusten führen.

Mini-Case: Von Like bis Sitzplatz

Fall	Garantie	Begründung
Like-Zähler	Eventual	Zähler zieht eine Sekunde später nach — Schaden gering
Warenkorb	Read-Your-Writes	eigene Änderungen sofort sichtbar, globale Verzögerung tolerabel
Sitzplatz-/Ticketbuchung	Strong	doppelte Vergabe = direkter fachlicher Schaden

Denkbewegung: erst der Fachschaden eines alten/widersprüchlichen Zustands → dann die passende Garantie → dann die technische Einstellung. **Leitfrage einordnen:** Wie teuer ist ein alter oder widersprüchlicher Read in diesem Fachkontext?

UE13 · NoSQL-Vergleich und Use Cases

Leitfrage: Welcher NoSQL-Typ passt zu welchem Problem?

Vier Vergleichsfragen

1. Wie sieht das **Datenmodell** aus?
2. Welche **Abfragen** sind natürlich?
3. Welche **Konsistenz-/Replikationslogik** ist typisch?
4. Wodurch **skaliert** der Typ besonders gut?

Gegenüberstellung

Typ	Natürliche Stärke	Leitfrage	Typische Grenze
Key-Value	sehr schneller Zugriff pro Schlüssel	Ist der Key bereits die Antwort?	komplexe Struktur bleibt außerhalb des Modells
Document	fachliche Einheit lesen und schreiben	Was gehört als Objekt zusammen?	Querbeziehungen werden aufwendig
Wide Column	geordnete Bereiche und große Tabellen	Welcher Row Key trägt den Zugriff?	spontane Kreuzabfragen sind schwierig
Graph	Beziehungen, Pfade, Nachbarschaften	Welche Kanten sind das Fachobjekt?	einfache Objektwelten brauchen oft kein Graphmodell

Kompakter Entscheidungsgang (4 Schritte)

1. **Dominierende Fachfrage** benennen.
2. Beschreiben, welche **Einheit** gemeinsam gelesen, geschrieben oder traversiert wird.
3. **Typ wählen**, dessen Modell diese Einheit natürlich trägt.
4. Knappe **Abgrenzung gegen mindestens eine plausible Alternative**.

Mini-Case Lernplattform (Multi-Model)

Bedarf	Typ	Begründung
Kursbeschreibung mit Modulen, Terminen, Materialien	Document	wird als zusammenhängende Einheit gelesen, bearbeitet, versioniert
Aktivitätsereignisse (Seitenaufrufe, Bearbeitungszeiten)	Wide Column	Großmengen, geordnet nach Nutzer/Kurs/Zeit, Bereichszugriffe
Beziehungen zwischen Studierenden, Gruppen, Empfehlungen	Graph	Nachbarschaften und Pfade sind das Erkenntnisinteresse

Multi-Model ist **nicht automatisch besser** — es erzeugt Integrations- und Betriebsaufwand. Aber: wenn eine Fachwelt mehrere deutlich unterschiedliche **Zugriffswelten** hat, kann mehr als ein Modell plausibel sein.

Warum Fehlwahl teuer wird: Sie fällt nicht sofort auf. Probleme zeigen sich später — umständliche Abfragen, redundante Ablage, unklare Konsistenzpfade, Performanceziele nur mit Zusatzkomplexität.

Klausursatz: „Dieses Modell ist hier ungünstig, weil der dominante Zugriff nicht natürlich getragen wird.“

BLOCK D — Verteilte Dateisysteme & Object Stores

UE14 · Motivation und Anforderungen an Distributed File Systems

Leitfrage: Warum brauchen wir verteilte Dateisysteme?

 Ein DFS ist **keine größere Festplatte** und **kein großes NAS**, sondern eine Antwort auf verteilte Last, große Datenmengen und Komponentenausfall.

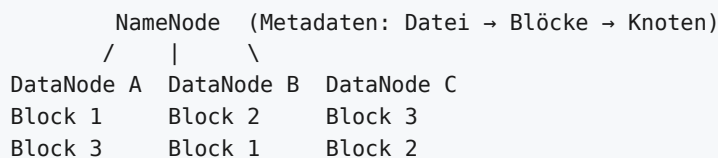
Wichtig: Unterscheidung **Kapazität** („mehr Speicherplatz“) vs. **Durchsatz** (wie schnell gelesen/verarbeitet werden kann).

Anforderungen an ein DFS

1. **Skalierung** — Speicher und Leistung wachsen durch Hinzufügen weiterer Knoten.
2. **Hoher Durchsatz** — große Datenmengen sequentiell und parallel lesen.
3. **Fehlertoleranz** — Knotenausfall macht die Datei nicht unbrauchbar.
4. **Metadatenorganisation** (oft übersehen) — welche Datei besteht aus welchen Blöcken, auf welchen Knoten? → hier wird die Rolle des NameNode verständlich.

Aspekt	Lokales File System	Distributed File System
Kapazität	an einen Rechner gebunden	über viele Knoten erweiterbar
Durchsatz	durch einzelne Maschine begrenzt	paralleles Lesen und Schreiben
Fehlertoleranz	Ausfall des Hosts ist kritisch	Replikation und Verteilung puffern ab
Verwaltung	lokale Metadaten und Pfade	zentral/koordiniert verwaltete Metadaten

HDFS-Grundmodell



Entscheidende Architekturidee: Trennung von Metadaten und Daten.

Warum große Blockgrößen?

- Weniger Metadaten pro Datei → geringerer Verwaltungsaufwand.
- Passt zu sehr großen, sequentiell gelesenen Dateien.
- ⇒ HDFS ist **nicht** für kleinste Einzeldateien mit minimaler Latenz optimiert, sondern für **großen Datendurchsatz**.

Datenlokalität als Leistungsprinzip

Rechenlogik möglichst zu den Daten bringen, nicht umgekehrt. Findet ein Rechenknoten einen Block bereits lokal/im selben Clusterbereich, reduziert das Netzwerkverkehr erheblich. Datenlokalität ist **keine Nebeneigenschaft, sondern eine echte Leistungslogik**.

✗ Wann ein DFS unpassend wäre

sehr kleine Dateien mit niedriger Einzellatenz · transaktionale Punktzugriffe · komplexe Fachabfragen über strukturierte Datensätze.

UE15 · HDFS-Architektur im Detail

Leitfrage: Wie arbeitet HDFS beim Lesen und Schreiben?

Die drei Rollen

Rolle	Zentrale Aufgabe	Typischer Fehler im Verständnis
Client	startet Lese- und Schreibvorgänge	wird wie ein interner Clusterknoten gelesen
NameNode	verwaltet Metadaten + Platzierungswissen	wird für den eigentlichen Datenspeicher gehalten
DataNode	speichert Blöcke, transportiert Daten	wird nur als passive Festplatte verstanden

★ Metadatenpfad vs. Datenpfad

- **Metadatenpfad:** Welche Datei besteht aus welchen Blöcken? Wo liegen sie? Welche DataNodes kommen für neue Replikate infrage? → **NameNode**
- **Datenpfad:** die eigentlichen Byte-Ströme, physisches Lesen/Schreiben von Blockinhalten → **DataNodes**

Write Path (Schrittfolge)

Client → NameNode → DataNode A → B → C

#	Schritt	Pfad
1	Client meldet beim NameNode eine neue Datei bzw. einen neuen Block an	Metadatenpfad
2	NameNode trifft Platzierungsentscheidung , nennt die Zielkette an DataNodes (Replikationsfaktor, Rack Awareness)	Metadatenpfad
3	Datenstrom startet vom Client zum ersten DataNode	Datenpfad
4	Block wird entlang der Replikationskette an weitere DataNodes weitergereicht	Datenpfad
5	DataNodes bestätigen; erst nach vollständiger Bestätigung der Kette gilt der Schreibvorgang als abgeschlossen	Datenpfad
6	Datei kann weitergeschrieben werden	—

Leitidee: Der NameNode **plant einmal**, danach läuft der Block als **Replikationsstrom** über die DataNodes. Replikation ist damit ein *kontrollierter Ablauf*, nicht eine lose Sammlung separater Kopiervorgänge.

Read Path (kürzer & asymmetrisch)

1. Client fragt NameNode: welche Blöcke gehören zur Datei, auf welchen DataNodes sind sie erreichbar? (*Metadatenpfad*)
2. Client liest Blockdaten **direkt** von einem geeigneten DataNode. (*Datenpfad*)

Asymmetrie: Beim **Schreiben** plant der NameNode **aktiv** (neue Speicherorte, Replikationsziele). Beim **Lesen** liefert er nur **Orientierungswissen**. **Merksatz:** *Der NameNode vermittelt den Weg, gelesen wird aber von den DataNodes.*

Replikation & Rack Awareness

- Replikation: Knotenausfall ≠ Datenverlust; defekte/unerreichbare Blöcke werden kompensiert.
- **Rack Awareness:** Lügen alle Replikate im selben Rack, könnte ein gemeinsamer Infrastrukturfehler mehrere Kopien zugleich treffen. Deshalb Verteilung über **verschiedene Ausfallzonen**. → Replikation ist nicht „mehr Kopien“, sondern **bewusste Verteilung von Risiko und Verfügbarkeit**.

Failure-Szenarien

Szenario	Reaktion
DataNode fällt aus	Heartbeats bleiben aus → System kann reagieren
Blöcke unterrepliziert	Replikation wird neu aufgebaut
Block als defekt erkannt	saubere Kopie aus vorhandenen Replikaten rekonstruiert

UE16 · HDFS: Eigenschaften, Fehlertoleranz und Grenzen

Leitfrage: Wo ist HDFS stark und wo liegen die Grenzen?

Fehlertoleranz als Architekturprinzip

Mechanismus	Beitrag	Worauf die Klausurantwort zielen sollte
Heartbeat	Ausfälle von DataNodes werden sichtbar	Erreichbarkeit und Reaktion des Systems
Block Report	Metadatenbild bleibt aktuell	Wissen über reale Blockverteilung
Replikation	Lesbarkeit und Neuaufbau trotz Ausfall	Verfügbarkeit und Risikoabfederung

Datenlokalität & Performance

Compute fern --Netzwerk--> Daten  teuer
 Compute nah → Block lokal nutzen  günstig

Deshalb wird HDFS über **Durchsatz** statt über **Latenz** beschrieben.

Die drei klassischen Grenzen

1. **Small Files** — sehr viele kleine Dateien erzeugen viel Metadatenaufwand, ohne die Stärke großer sequentieller Blockzugriffe auszuspielen. *Die Dateiform arbeitet gegen die Architektur.*
2. **Niedrige Einzellatenz** — HDFS ist nicht für schnelle Punktreaktionen auf viele kleine Anfragen optimiert.
3. **Random Writes** — viele kleine, unvorhersehbare Aktualisierungen passen schlecht zu großen, geordneten Datenströmen.

Diese Grenzen sind **keine Maken**, sondern der Schlüssel zur Wahlkompetenz. Ein System, das scheinbar alles kann, lässt sich nicht präzise begründen.

Workload-Passung

 große Logdateien, Sensordaten, ETL-Pipelines, Scan-Logik, verteilter Durchsatz, Batch  viele kleine Dateien, sehr schnelle Einzelzugriffe, transaktionale Update-Muster

Kompakter Entscheidungsrahmen

1. **Dateiform:** wenige große Dateien oder sehr viele kleine Einheiten?
2. **Zugriffsmuster:** große Scans oder häufige Punktzugriffe?
3. **Leistungsziel:** Durchsatz oder niedrige Latenz?

Mini-Case: Archivplattform oder Dateimüll?

Fall	Beurteilung
Täglich große Log-/Ereignisdateien, später Batch-Jobs	 Architektur greift natürlich
Millionen kleiner Dokumente mit häufigen Einzelzugriffen	 Dateiform arbeitet gegen die Systemlogik

Kernerkenntnis: Datenmenge allein ist kein hinreichendes Kriterium.

UE17 · Object Stores

Leitfrage: Was macht Object Stores so skalierbar und preiswert?

Das Grundmodell

Baustein	Funktion	Worauf man achten sollte
Bucket	logischer Namensraum	⚠ nicht mit einem echten Verzeichnis verwechseln
Objekt	gespeicherte Einheit (Nutzdaten + Schlüssel)	Schlüsselzugriff steht im Zentrum
Metadaten	Beschreibung und Verwaltung (Typ, Zeitstempel, Tags, Versionen)	helfen bei Nutzung, Suche, Lifecycle

Flacher Namespace statt Verzeichnisbaum

Ein Schlüssel wie `logs/2026/04/17/app.json` **sieht** hierarchisch aus, bleibt aber eine **logische Struktur im Schlüssel** — kein tiefes Dateisystem mit Verzeichnissemantik und Pfadoperationen.

Der Tausch: weniger Dateisystemsemantik ⇔ hohe Plattformtauglichkeit.

Die API-Logik

```

PUT    Objekt schreiben/hochladen
GET    Objekt anhand des Schlüssels liefern
LIST   Objekte in einem Bucket / unter einem Prefix finden
DELETE Objekt entfernen
  
```

LIST ist begrifflich besonders wichtig — im Data-Lake-Kontext die **Scharnieroperation** zwischen Speicherung und späterer Verarbeitung. ⚠ LIST ist in großen Namensräumen **nicht automatisch billig oder sofort** → bewusst mit Schlüsseln, Prefixen und Metadaten umgehen.

★ Durability ≠ Verfügbarkeit

- **Durability:** Wie unwahrscheinlich ist es, dass Daten **verloren gehen**?
- **Verfügbarkeit:** Wie oft ist der Dienst **erreichbar und nutzbar**? Ein Speicher kann sehr hohe Durability besitzen und dennoch andere Verfügbarkeitszusagen haben.

Use Cases

✅ Data Lakes, Archive, Backups, Medieninhalte, Exportdaten, Rohdaten in Analyseplattformen, Plattformwelten ❌ strenge Datei-/Verzeichnissemantik, sehr dateisystemnahe Interaktion

Warum Object Stores so gut in Plattformen passen

Speicher ist hier nicht für **eine** Anwendung, sondern **gemeinsame Plattformbasis**: verschiedene Teams schreiben Rohdaten, andere lesen sie mit anderen Tools. Kleine stabile API + klare Objekte passen zu dieser Offenheit besser als ein tiefes Dateisystemmodell.

Merksatz: *Object Stores sind stark, weil sie bewusst **weniger** versprechen als ein Dateisystem — aber dieses reduzierte Versprechen sehr groß skaliert und robust als Plattdienst einlösen.*

Der Schlüssel strukturiert die logische Ordnung des Namensraums, beeinflusst Listenoperationen und bestimmt mit, wie gut spätere Nutzungsszenarien passen — analog zum **Row Key** in Wide-Column-Welten.

UE18 · DFS vs. Object Store und moderne Architekturen

Leitfrage: Wie entscheide ich zwischen DFS und Object Store in modernen Datenplattformen?

Fünf Vergleichsleitfragen

In welchem **Modell** wird gespeichert? · Wie wird **zugriffen**? · Welche **Leistungsziele** dominieren? · Wie sieht die **Betriebs-/Plattformlogik** aus? · Welche **Kosten- und Governance-Folgen** sind zu erwarten?

Kriterium	DFS (HDFS)	Object Store
Modell	Datei- und Blocklogik	Objekt- und API-Logik
Zugriff	dateisystemnah, pfadorientiert	schlüssel- und API-orientiert
Stärke	Compute-Nähe, Batch-Logik, Datenlokalität	Plattformspeicher, Skalierung, Wirtschaftlichkeit
Grenze	Small Files, Plattformoffenheit	Dateiillusion und bestimmte Pfaderwartungen

Organisatorische Lesart: DFS-Denken ist näher an **integrierter Speicher- und Verarbeitungsperspektive**. Object-Store-Denken passt zu **entkoppelten Plattformen** mit mehreren Verarbeitungseingängen.

Data Lake — ein Architekturprinzip, keine Produktkategorie

Rohdaten zentral und **möglichst offen** ablegen, sodass verschiedene Teams und Verarbeitungssysteme später darauf zugreifen. **Speicher und Compute werden deutlicher getrennt.**

Quellen → Storage Layer → Compute

Lakehouse — als Konzept

Verbindet **Offenheit und Speicherbasis eines Data Lake** mit **mehr Struktur, Tabellenlogik und Governance**.

⚠ Das Lakehouse ist **keine dritte Speichergrundform** neben DFS und Object Store, sondern eine **stärker organisierte Architektur auf einer Lake-nahen Speicherbasis**.

Entscheidungsrahmen (4 Schritte)

1. Welche Art von Zugriff dominiert — scan-orientierte Batch-Last oder offene Plattformnutzung?
2. Welche Modelllogik passt — **Datei** oder **Objekt**?
3. Wie wichtig sind Compute-Nähe, Wirtschaftlichkeit, Governance?
4. Welche Alternative wäre plausibel, und warum ist sie hier schwächer?

📦 Mini-Case: Batch-Cluster oder Plattformbasis?

Fall	Wahl	Begründung
Stark batch-orientierter Analysecluster, enge Kopplung Speicher↔Rechenjobs	DFS	Compute-Nähe, Datei-/Blocklogik, verteilte Verarbeitung
Offene Rohdatenbasis für mehrere Teams, verschiedene Engines	Object Store	Plattformlogik, Entkopplung Compute/Storage, organisatorische Offenheit

Beide Fälle haben große Datenmengen — nicht die Datenmenge entscheidet, sondern welche Architekturidee die Nutzung besser trägt.

Warum entkoppeln? Wenn viele Verarbeitungssysteme auf dieselbe Datenbasis zugreifen sollen, oder wenn Speicherkapazität und Rechenleistung **unterschiedlich wachsen**.

Beispiel-Klausursatz: „Für diesen Use Case passt ein Object Store besser, weil die Speicherbasis von der späteren Verarbeitung entkoppelt werden soll und mehrere Engines über eine offene Plattform auf dieselben Rohdaten zugreifen.“

BLOCK E — Verarbeitung

UE19 · Hadoop-Ökosystem im Überblick

Leitfrage: Wie greifen Storage und Processing im Hadoop-Ökosystem ineinander?

Komponente	Aufgabe	Worauf in der Klausur zu achten ist
HDFS	verteilte Speicherung	Datenbasis statt Rechenlogik
YARN	Ressourcen und Container planen	keine Datenhaltung , sondern Laufzeitkoordination
MapReduce	Batch-Verarbeitung ausführen	Datenfluss statt Produktdetail
Metastore	Tabellen- und Schemainformationen halten	Abstraktion über Dateien und Formate

Ablauf: Vom Job zur Verarbeitung

Job → YARN (Ressourcen/Container) → MapReduce (Verarbeitung) → HDFS (Daten)

1. Verarbeitungsanfrage wird formuliert/angestoßen.
2. **YARN** koordiniert Ressourcen, stellt Laufzeitumgebungen/Container bereit.
3. **MapReduce** verteilt Aufgaben auf konkrete Rechenknoten.
4. Daten liegen währenddessen bereits in **HDFS**.

Drei Sichten auf dieselben Daten

Sicht	Was man sieht
Speichersicht	Dateien in HDFS
Verarbeitungssicht	dieselben Dateien in einem Joblauf eingebunden, gelesen, verteilt, als Ausgabe abgelegt
Nutzungssicht	dieselben Dateien über Metadaten als Tabelle oder Partition

→ **Nicht drei Datenwelten, sondern drei Perspektiven auf dieselbe materielle Basis.**

⊘ Typische Fehlvorstellungen

1. „HDFS ist bereits das gesamte Hadoop-System.“ → unterschlägt Verarbeitungs- und Ressourcenebene.
2. „YARN verwaltet Daten.“ → YARN verwaltet **Rechenkapazität**, keine Inhalte.
3. „Metastore ist eine eigene physische Datenhaltung.“ → Er ist eine **logische/organisatorische Funktion über vorhandenen Dateien**.

💡 Warum das über Hadoop hinaus trägt

Auch moderne Plattformen trennen: **dauerhafte Speicherung · Ressourcensteuerung · Rechenlogik · Metadatenkatalog**. Die Namen ändern sich, die **Denkfigur bleibt stabil**. **Robustes Raster**: Was speichert hier? Wer plant? Wer verarbeitet? Wer beschreibt?

UE20 · HBase als storage-nahes System

Leitfrage: Wie funktioniert HBase als Wide-Column-Store auf HDFS?

Die Schichtung

Ebene	Rolle	Didaktischer Fokus
HDFS	verteilte Speicherbasis	HBase steht nicht ohne Plattform da
HBase	schlüssel- und familiennahes Datenmodell	Zugriff folgt RowKey und Column Family
Anwendung	liest/schreibt über HBase-Logik	kein direkter Dateizugriff als Hauptmodell

Datenmodell

```
RowKey: sensor42#2026-04-17T10
├─ meta   : standort | typ
├─ values : temperatur | druck
└─ flags  : alarm | qualitaet
```

Der RowKey bestimmt drei Dinge

1. Welche **Entitäten/Zeitbereiche** direkt adressierbar werden.
2. Die **Ordnung** von Zeilen → und damit Bereichszugriffe.
3. Die **Lastverteilung** im System (ungünstiger RowKey → Hotspots).

Column Families sind eine Speicherentscheidung, kein Kosmetikmerkmal: Werte mit gemeinsamer Zugriffsdynamik zusammen, Werte mit deutlich anderer Dynamik getrennt.

Von der Fachfrage zum Zugriffspfad

Kette: Die Fachfrage bestimmt den RowKey → der RowKey bestimmt Ordnung und Zugriffsweg → daraus ergibt sich, ob HBase für den Fall natürlich oder unnatürlich ist.

✓/✗ Stärken und Grenzen

✓ schlüssel- und bereichsnahe Zugriffe · große Tabellen · Random Reads/Writes über einen klaren RowKey · **Zeitreihen** ✗ freie analytische Abfragen · komplexe Joins · SQL-nahe Ad-hoc-Welt

⚠️ HBase ≠ verteilte relationale Datenbank

Die Tabellenform ist **kein Versprechen auf relationale Flexibilität**, sondern ein **Ordnungsrahmen für RowKeys und Column Families**. Relationale Systeme optimieren viele *unterschiedliche* Anfragen über allgemeine Schemata, Indizes und Operatoren. HBase optimiert eine **engere, aber sehr leistungsfähige Zugriffswelt**.

📦 Mini-Case: Sensordaten für Betriebsmonitoring

Frage: „Welche Werte liegen für Sensor X im Zeitfenster Y vor, und welche Alarme traten auf?“ → RowKey trägt **Sensor + Zeit**, Column Families trennen **Messwerte / Metadaten / Statussignale**. ✅ natürliche Passung.

UE21 · MapReduce als Batch-Paradigma

Leitfrage: Wie funktioniert MapReduce als geordnete Batch-Verarbeitung?

Die Pipeline

Map	→	Shuffle	→	Reduce
lokale		neu gruppieren		Endergebnisse
Teilresultate		und ordnen		verdichten

Phase	Funktion	Didaktische Aussage
Map	Eingaben lokal lesen und in Teilresultate überführen	Verteilung beginnt datennäher als viele vermuten
Shuffle	gleiche Schlüssel neu zusammenführen und ordnen	Netzlast und Umordnung sind der zentrale Kostenpunkt
Reduce	zusammengehörige Werte verdichten	das Endergebnis entsteht erst am Schluss der Pipeline

⚠️ **Map ist nicht einfach „Parallelität“.** Ihr Sinn: lokale Daten in eine Form bringen, mit der sich später sinnvoll weiterarbeiten lässt. ⚠️ **Reduce ist nicht nur Summieren.** Auch Gruppierung, Konsolidierung, andere Formen der Verdichtung.

Datennahe Verarbeitung

Nicht „viele Rechner sind schneller“, sondern: **Rechenarbeit möglichst nah an den Daten starten.**
ABER: Spätestens im Shuffle müssen Ergebnisse neu zusammenfinden → verteilte Verarbeitung bedeutet **immer auch Koordination und Transport.**

MapReduce ist ein Batch-Modell

Klar abgegrenzte Datenmenge · klarer Start · klarer Lauf · Ende mit Resultaten. ✅ **Stark bei:** großen Volumina, klaren Schlüsseln, reproduzierbaren Abläufen. ❌ **Schwierig bei:** iterativen Algorithmen, sehr niedrigen Latenzanforderungen, Verarbeitungslogik als Netz vieler Abhängigkeiten statt fester Zweiphasen-Struktur.

📦 Mini-Case: Fehlercodes in Tageslogs

Jeder Datenblock lokal gelesen → jede Logzeile → Paar (Fehlercode, 1) → Shuffle führt gleiche Codes zusammen → Reduce bildet Gesamtzahlen. ✅ Groß, strukturiert, keine interaktive Reaktion nötig.

💡 Warum das die Architekturpraxis bis heute prägt

Viele moderne Datenpipelines unterscheiden weiterhin: **lokale Vorverarbeitung → teure Zusammenführung nach Schlüsseln → spätere Verdichtung/Materialisierung.** Die Technik ändert sich, der Grundgedanke des Datenflusses bleibt.

UE22 · Spark und DAG-basierte Verarbeitung

Leitfrage: Warum sind DAG-basierte Engines flexibler als das starre MapReduce-Modell?

DAG = Directed Acyclic Graph

Verarbeitung als **Folge von Schritten mit klaren Abhängigkeiten**; ein späterer Schritt baut auf früheren auf, keine Rückschleife im Ausführungsplan.

lesen → filtern → transformieren → gruppieren → Ergebnis

Der Blick wechselt **von einzelnen Jobs auf ganze Arbeitspläne**. Das Modell denkt in **Abhängigkeiten zwischen Operationen** statt in einer fixen Folge von Phasen.

Transformation vs. Action

Begriff	Bedeutung	Didaktische Kernaussage
Transformation	baut den Arbeitsplan weiter auf	noch kein finales Ergebnis, sondern geplanter Datenfluss
Action	löst Ausführung / Materialisierung aus	jetzt wird der Plan konkret gebraucht
Lazy Evaluation	plant zuerst, rechnet später	Optimierung wird vor der Ausführung möglich

⚠ Spark ist NICHT „MapReduce, nur schneller“

Der tiefere Unterschied: **ein reichhaltigeres Planungsmodell für Abhängigkeiten**. Mehrere aufeinanderfolgende Operationen können als zusammenhängender Graph gelesen und **gemeinsam optimiert** werden, ohne jeden Schritt in eine neue starre Jobgrenze zu pressen.

Flexibilität ist keine Magie. Sie entsteht aus reicherem Arbeitsplan + späterer Auslöselogik + besserer Abstimmung der Verarbeitungswege. Auch Spark bleibt abhängig von Datenmengen, Netzlast, Partitionen, Ressourcenabstimmung.

Batch vs. Streaming (begrifflich sauber)

- **Batch** = Verarbeitung einer **abgeschlossenen** Datenmenge.
- **Streaming** = Verarbeitung eines **fortlaufenden Stroms** von Ereignissen.

⚠ **Streaming ≠ garantierte Echtzeit.** Auch dort bleiben Latenz, Ressourceneinsatz und Arbeitsplan zentrale Fragen.

📦 Mini-Case: Mehrstufige Klickstrom-Analyse

Ereignisse filtern → auf Nutzergruppen abbilden → Kennzahlen pro Gruppe zusammenfassen → speichern. *Filtern, Zuordnen, Gruppieren* = **Transformationen** (Plan). *Speichern/Zählen am Ende* = **Action** (Ausführung).

BLOCK F — Integration, Transfer, Klausur

UE23 · End-to-End-Architektur

Leitfrage: Wie greifen Quellen, Speicher, Verarbeitung und Nutzung zusammen?

Quellen → Storage Layer → Processing → Zugriff
 HDFS/Object Store MapReduce/
 Spark

Ebene	Leitfrage	Typische Systeme
Quellen	Wo entstehen Daten und in welcher Form kommen sie an?	Logs, Dateien, Transaktionen, Events
Storage	Wie werden Daten stabil und verteilt bereitgestellt?	HDFS, Object Store
Processing	Wie werden Rohdaten geordnet zu Ergebnissen verarbeitet?	MapReduce, Spark
Zugriff	Wie werden Ergebnisse genutzt oder weitergegeben?	Analyse, Reporting, Weiterverarbeitung

Merksätze:

- **Quellen erzeugen Heterogenität, Speicher erzeugt Stabilität, Verarbeitung erzeugt Umformung, Zugriff erzeugt Nutzung.**
- Quellen sind **Datentypen**, aber noch keine **Architekturebene**.
- **Verarbeitung ≠ Speicherung.** Spark/MapReduce halten *keine* dauerhafte, stabile Datengrundlage bereit — ihre Rolle ist Umformung.
- **Zugriff ≠ Rohdatenhaltung.** Die Nutzungsschicht braucht *verarbeitete, verstehbare Ergebnisse*.

Mini-Case „CampusShop“

Klickdaten (Nachfrage) · Bestellungen (Kauf) · Produktbilder (unstrukturiert) → stabile Ablage (HDFS oder Object Store) → Verarbeitung (Nachfragecluster, Tagesberichte, Sortimentsauswertungen; Batch-Pipeline oder DAG) → sichtbare Nutzung (Berichte, Steuerungsentscheidungen).

Eine gute End-to-End-Architektur ist nicht gut, weil sie viele Bausteine hat, sondern weil jede Schicht eine saubere Rolle trägt und die Übergänge nachvollziehbar bleiben.

Typische Rollenvermischungen

- HDFS/Object Store für eine **Analysemaschine** halten (Speicher ≠ Auswertung).
- Spark so lesen, als löse es auch **dauerhafte Datenhaltung**.
- Quellen mit Architekturbausteinen verwechseln.
- Zugriffsschicht (Berichte, Visualisierung) als technische Nebensache lesen — sie bildet **den Zweck der gesamten Architektur** ab.

UE24 · Vergleich und Entscheidung

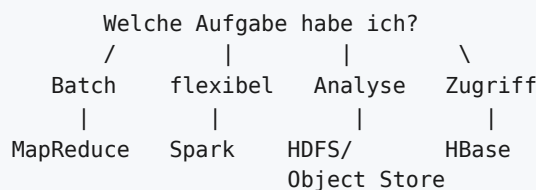
Leitfrage: Wie ordne ich HDFS, Object Store, MapReduce, Spark und HBase begründet ein?

Die vier Vergleichskriterien

1. **Zugriffsmuster** — große sequentielle Läufe vs. gezielte wiederholte Einzelzugriffe
2. **Latenz** — schnelle wiederholte Verfügbarkeit vs. länger laufender Batch akzeptabel
3. **Datenform** — Speicherbasis vs. zugriffsorientierte Struktur
4. **Flexibilität** — feste Pipeline vs. reicherer Arbeitsplan

★ **Analyse und Zugriff sind nicht dasselbe.** Ein *Analyseproblem* fragt nach Mustern/Zusammenhängen/Kennzahlen über große Mengen. Ein *Zugriffsproblem* fragt nach einzelnen Datensätzen oder begrenzten Bereichen.

Der Vergleichsraum



System	Typische Stärke	Typische Grenze
HDFS	verteilte, große Speicherbasis für Batch-nahe Nutzung	nicht für schnellen Einzeldatenzugriff gedacht
Object Store	lose gekoppelte, breit nutzbare Speicherbasis	nicht selbst die Verarbeitung oder Fachlogik
MapReduce	klare Batch-Pipeline für große Läufe	sperrig bei reicheren Verarbeitungswegen
Spark	flexiblerer Arbeitsplan für Analyse	nicht die Antwort auf jede Zugriffsfrage
HBase	gezielter Zugriff auf große strukturierte Datenmengen	nicht primär für freie Großanalyse

⊘ Typische Fehlentscheidungen

- HDFS für **schnellen Zugriff auf einzelne Datensätze** einplanen.
- HBase für **freie Großanalyse** verwenden.
- **Spark mit Speicherung** verwechseln.
- MapReduce als Antwort auf **jede** Form von Analyse lesen.

Prüfungslogik: Eine Entscheidung ist dann falsch, wenn das gewählte System die dominante Aufgabe nicht gut trägt.

📦 Mini-Case: Reporting + Kundenzugriff (CampusShop)

Bedarf	Problemtyp	Antwort
Tägliche Verkaufs-/Nachfrageberichte	Analyseproblem	Spark oder MapReduce auf HDFS/Object-Store-Basis
Support schlägt einzelne Bestellungen schnell nach	Zugriffsproblem	HBase

Richtige Antwort ist gerade NICHT, ein einziges System für alles zu suchen. Mehrere Systeme sind aber nicht automatisch besser — nur wenn unterschiedliche dominante Aufgaben tatsächlich unterschiedliche Stärken verlangen.

Regel: Nicht fragen, welches System am meisten kann, sondern welches System für welche Teilaufgabe die beste Passung besitzt.

UE25 · Architektur analysieren (Diagnose)

Leitfrage: Wie erkenne ich, in welcher Ebene einer Architektur ein Problem eigentlich liegt?

Diagnose vor Lösung. Die erste Frage lautet nicht „welches System?“, sondern „**welche Ebene?**“

Symptom → Ebene

Symptom	Naheliegende Ebene	Leitfrage der Diagnose
Langsame Berichte	Processing	Ist die Verarbeitung passend zur Analyseaufgabe organisiert?
Langsamer Zugriff (einzelne Datensätze)	Zugriff	Geht es um gezielte Datenabfragen statt freier Großanalyse?
Daten nicht nutzbar / uneinheitlich	Quelle	Kommen die Daten überhaupt in klarer, anschlussfähiger Form an?
Schlecht passende Datenbasis	Storage	Wird die Datenhaltung so bereitgestellt, dass spätere Nutzung sinnvoll möglich ist?

Mini-Case: CampusShop mit drei Störungen

1. Verkaufsberichte spät fertig → **Processing-Problem**
2. Support braucht lange, um Bestellungen zu finden → **Zugriffsschicht**
3. Quellsysteme liefern Produktattribute uneinheitlich → **Quellenproblem**

Die Denkleistung ist die Zerlegung in Architekturrollen — nicht die sofortige Nennung eines Systems.

Typische Fehldiagnosen

1. Jedes sichtbare Problem vorschnell dem **Storage** zuschreiben (weil Storage in Architekturbildern zentral sitzt).
2. **Quellprobleme zu spät bemerken** — kommen Daten bereits uneinheitlich an, kann spätere Verarbeitung nur begrenzt helfen.
3. Suche nach **einer einzigen Ursache** — verschiedene Symptome können verschiedene Ebenen betreffen.

Antwortreihenfolge: Beobachtung → Ebene → Begründung → (erst danach) Systemlogik.

UE26 · Klausurtraining

Die drei Operatoren

Operator	Was erwartet wird	Typische gute Antwortform
Einordnen	Rolle, Ebene oder Kategorie sichtbar machen	Kategorie nennen + kurz mit Aufgabenbezug absichern
Vergleichen	zwei Gegenstände entlang desselben Kriteriums gegenüberstellen	gemeinsames Kriterium → Unterschied → Einordnung
Begründen	Entscheidung aus Aufgabe und Kriterium ableiten	Aufgabe benennen → Kriterium nennen → Passung sprachlich herstellen

✗ Zwei isolierte Merkmallisten erfüllen den Operator *vergleichen* **nicht**. ✗ Ein bloßer Produktname erfüllt den Operator *begründen* **nicht**.

★ Das Antwortgerüst (4 Schritte)

1. Aufgabe verstehen → 2. Kriterium nennen → 3. System einordnen → 4. Begründung formulieren

Schwache → starke Formulierungen

✗ Schwach	✓ Stark
„Spark ist besser, weil es moderner ist.“	„Spark passt besser, wenn die Aufgabe eine flexible Folge mehrerer Verarbeitungsschritte über große Datenmengen verlangt.“
„HBase ist für Zugriff.“	„HBase ist zugriffsnah einzuordnen, weil hier gezielte Datenabfragen und nicht freie Großanalyse im Vordergrund stehen.“
„Object Store, weil modern.“	„Object Store, weil die Speicherbasis von der Verarbeitung entkoppelt werden soll und mehrere Engines auf dieselben Rohdaten zugreifen.“

UE27 · Wiederholung und Integration

★ Das Gesamtmodell des Kurses

Storage Processing Zugriff
HDFS / Object Store → MapReduce / Spark → HBase / Nutzung

Rolle	Zentrale Systeme	Leitfrage
Storage	HDFS, Object Store	Wie werden Daten dauerhaft und verteilt bereitgestellt?
Processing	MapReduce, Spark	Wie werden Daten geordnet verarbeitet und ausgewertet?
Zugriff	HBase, Nutzungsschicht	Wie werden Datensätze oder Ergebnisse gezielt genutzt?

Die wichtigste Verdichtung: Storage ist nicht Processing, Processing ist nicht Zugriff.

Die zwei großen Vergleiche

1. **HDFS vs. Object Store** — beide im *Speicherraum*, Unterschied in der **Einbettung in Architekturen** (clusternahes Dateidenken vs. lose serviceartige Speicherbasis).
2. **MapReduce vs. Spark** — beide im *Verarbeitungsraum*, Unterschied im **Verarbeitungsmodell** (feste Pipeline vs. reichere flexible Folgen).

🚫 Die drei häufigsten Fehlzuordnungen

1. **Spark als universelle Lösung lesen** — es ist eine starke Verarbeitungsschicht, aber keine Speicherbasis und kein zugriffsnahes System.
2. **HBase in den Speicherraum ziehen**, nur weil es große Datenmengen verwaltet — fachlich richtig ist die Einordnung über das **Zugriffsmuster**.
3. **HDFS und Object Store als „dasselbe in alt/modern“** — es sind **unterschiedliche Antworten im selben Speicherraum**.

🔑 Die Merkhilfe für die Prüfung

Rolle vor Produkt · Kriterium vor Wertung · Passung vor Schlagwort

OFFIZIELLE BEISPIELKLAUSUR (25 Punkte / 6 Aufgaben)

Hinweis des Dozenten: Diese Beispielfragen zeigen **Aufgabentypen, Operatoren und die Logik der erwarteten Antworten**; sie bilden nicht die spätere Klausur ab. Spark und Graph-DBs sind in den Beispielen ausgespart.

Nr.	Thema	Punkte
1	Zugriffsmuster zuordnen	4
2	NoSQL-Typ begründet wählen	5
3	Storage-Paradigma einordnen	3
4	HDFS vs. Object Store	4
5	Mischarchitektur in einer Lernplattform	4
6	HDFS-Architektur: Write Path und Rollen	5
Σ		25

Frage 1 (4 P) — Zugriffsmuster zuordnen

Ordnen Sie vier Szenarien einem Zugriffsmuster zu (*Point Lookup, Range Scan, Append, Object Read*) und begründen Sie jede Zuordnung in einem Satz.

Szenario	Zugriffsmuster	Begründung
a) Live-Preisabfrage in einem Onlineshop	Point Lookup	Key-basierter Einzelzugriff mit Erwartung an sofortige Antwort
b) Tagesreport über alle Verkäufe eines Monats	Range Scan / Batch Read	Großer Zeitraum wird sequentiell gelesen und aggregiert
c) Eingehende Sensorwerte einer Produktionsanlage	Append (mit späteren Range Scans)	Kontinuierlicher Eventstrom, angehängt und später zeitlich ausgewertet
d) Abruf eines archivierten Vertragsscans	Object Read	Großes, unveränderliches Objekt, selten und als Ganzes gelesen

Frage 2 (5 P) — NoSQL-Typ begründet wählen

Ein Streaming-Dienst speichert pro Nutzer ein umfangreiches Profil mit variablen Präferenzen, Watchlists und Einstellungen. Eine Profilseite wird als Ganzes geladen.

a) Wahl: Document

b) Begründung:

Stärke	Grenze	Preis der Fehlwahl
Das Profil wird als fachliche Einheit gelesen und geschrieben — genau die Stärke von Document Stores	Querbeziehungen zwischen Profilen (z.B. „Freunde mit ähnlichen Vorlieben“) werden im Document-Modell aufwendig	Bei Key-Value ginge die Feldzugriffsstruktur verloren; Teilabfragen würden unnötig schwer

c) **Alternative Key-Value:** schneller Zugriff über die Nutzer-ID, aber die innere Struktur des Profils bliebe außerhalb des Modells — selektive Updates einzelner Felder ohne Anwendungssemantik wären nicht möglich.

Frage 3 (3 P) — Storage-Paradigma einordnen

Backups von Server-Snapshots: sehr große, unveränderliche Dateien, seltener Zugriff, hohe Anforderung an Durability.

a) Wahl: Object Storage

b) Zwei Eigenschaften:

- **Adressierung:** flacher Namespace mit ID-basiertem Zugriff über API/HTTP — ideal für sehr viele große Objekte.
- **Zugriffsmuster + Durability:** unveränderliche, selten gelesene Objekte mit hoher Durability sind genau das Einsatzfeld von Object Stores.

c) **Ausschluss Block Storage:** Block ist auf **Random Writes** und **niedrige Latenz für aktive Volumes** ausgelegt; für selten gelesene, unveränderliche Großdateien wäre das überdimensioniert und teuer.

Frage 4 (4 P) — HDFS vs. Object Store

a) Vergleichstabelle

Kriterium	HDFS	Object Store
Zugriffsmuster	sequentielle Batch-Verarbeitung großer Blöcke, eng mit Compute gekoppelt	direkter Objektzugriff über API/HTTP, viele unabhängige Reads/Writes
Flexibilität	eng an Hadoop-Ökosystem gebunden, Speicher und Compute zusammen	von Compute entkoppelt, mehrere Engines können denselben Datenraum nutzen

b) Beurteilung von „HDFS ist veraltet, seit es Object Stores gibt“: Die Aussage ist **falsch**. Beide gehören in den Speicherraum, verfolgen aber unterschiedliche Kopplungs- und Nutzungsideen — HDFS ist auf enge Compute-Kopplung und Batch ausgelegt, Object Store auf entkoppelte, weltweite Zugriffe.

Frage 5 (4 P) — Mischarchitektur Lernplattform

a)

Bedarf	Typ	Begründung
Kursbeschreibungen (Module, Termine, Materialien; als Ganzes geladen/bearbeitet/versioniert)	Document	Ein Kurs ist eine zusammenhängende fachliche Einheit — die natürliche Stärke des Document-Modells
Aktivitätseignisse (Seitenaufrufe, Bearbeitungszeiten; große Zahl, sequentiell nach Nutzer/Kurs/Zeit ausgewertet)	Wide Column	Großmengen, geordnete Ereignisse mit Row Key nach Nutzer/Kurs/Zeit → effiziente Bereichszugriffe

b) Die beiden Bedarfe haben **unterschiedliche Zugriffswelten** — *fachliche Einheit lesen vs. geordnete Bereiche scannen*. Typen sind Antworten auf genau solche Zugriffswelten; ein einzelner Typ würde in einem der beiden Aspekte deutlich verlieren.

Frage 6 (5 P) — HDFS-Architektur: Write Path und Rollen

Ein Team äußert: „Der NameNode ist dann unser Speichercluster.“

a) Die drei Hauptrollen

Client	NameNode	DataNode
Startet Lese-/Schreiboperationen; ist kein interner Clusterknoten , sondern die anfragende Seite	Verwaltet Metadaten (Datei-, Block-, Platzierungsinformationen) und trifft Platzierungsentscheidungen; speichert selbst keine Blockdaten	Speichert die eigentlichen Blöcke und transportiert die Datenströme zwischen Client und Replikationskette

b) Write Path → siehe Tabelle in UE15 (Schritte 1–6 mit Metadaten-/Datenpfad-Kennzeichnung).

c) Beurteilung: Die Aussage ist **unpräzise**. Der NameNode *ordnet* die Datenwelt (Metadaten und Platzierungswissen), gespeichert werden die Blöcke aber auf den **DataNodes** — Metadatenebene und Datenebene müssen architektonisch getrennt gedacht werden.

SELBSTTEST — Fragen zum Abfragen

▼ Block A

1. Nenne die fünf Ebenen des Big-Data-Stacks in der richtigen Reihenfolge.
2. Erkläre den Unterschied zwischen Latenz und Durchsatz an einem Beispiel.
3. Ordne zu: OLTP / OLAP / Batch / Streaming — jeweils Ziel und Speicherfolge.
4. Warum ist „JSON ist eine Datenbank“ ein Denkfehler?
5. Leite für „Klickereignisse eines Onlineshops“ die Speicherlogik über die vollständige Kette her.
6. Range- vs. Hash-Sharding: Nenne je Stärke, Risiko und typischen Fit.
7. Was ist ein Hotspot, welche zwei Formen gibt es, und nenne drei Gegenmaßnahmen mit ihrem Preis.
8. Erkläre Consistent Hashing und den Zweck virtueller Nodes.
9. Warum ist Replikation kein Backup?
10. Was besagt CAP wirklich — und was besagt es *nicht*?
11. Trenne ACID und CAP in je einem Satz.
12. Warum ist Read-Your-Writes didaktisch wichtig?

▼ Block B

13. Nenne die fünf Storage-Paradigmen mit Adressierung, Stärke und Grenze.
14. Warum ist ein Bucket kein Ordner?
15. Vergleiche Relational / Dokument / Graph / Cube entlang Kernlogik, Stärke, Preis.
16. Schema on Write vs. Schema on Read: Vorteil und Preis.
17. Was ist Denormalisierung — und welchen Tausch beschreibt sie genau?
18. Nenne die fünf Auswahlkriterien und je einen Zielkonflikt.
19. Warum sind Kosten mehr als Hardwarepreise?
20. Nenne die drei typischen Begründungsfehler bei Speicherentscheidungen.

▼ **Block C**

21. Nenne die drei Treiber für NoSQL.
22. Wofür steht BASE und wie grenzt es sich von ACID ab?
23. Nenne die drei häufigsten NoSQL-Fehlurteile und korrigiere sie.
24. Key-Value: Welche drei Operationen bilden die API, und was ist die wichtigste Rückfrage bei jedem Use Case?
25. Wann kippt die Passung eines Key-Value Stores?
26. Document Store: Wann Einbettung, wann Referenz? Vier Entscheidungsfragen.
27. Warum brauchen auch Document Stores Indizes?
28. Erkläre Row Key, Column Family und Cell.
29. Was ist der Unterschied zwischen Wide Column Store und analytischem Column Store?
30. Nenne zwei Row-Key-Fehlmuster.
31. Erkläre die LSM-Idee und warum sie zu schreibintensiven Szenarien passt.
32. Erkläre die Quorum-Regel $R + W > N$. Was gilt bei $N=5$, $W=2$, $R=2$?
33. Warum ist Last-Write-Wins „einfach, aber nicht automatisch fachlich gut“?
34. Ordne zu: Like-Zähler / Warenkorb / Sitzplatzbuchung → Konsistenzgarantie + Begründung.
35. Nenne die vier NoSQL-Typen mit Leitfrage und typischer Grenze.

▼ **Block D**

36. Nenne die vier Anforderungen an ein DFS.
37. Warum arbeitet HDFS mit großen Blockgrößen?
38. Erkläre Datenlokalität als Leistungsprinzip.
39. Beschreibe den HDFS Write Path in sechs Schritten mit Metadaten-/Datenpfad-Kennzeichnung.
40. Warum ist der Read Path asymmetrisch zum Write Path?
41. Was leistet Rack Awareness und warum?
42. Nenne die drei Fehlertoleranz-Mechanismen und ihren jeweiligen Beitrag.
43. Erkläre das Small-Files-Problem — warum genau ist es problematisch?
44. Nenne die vier Bausteine des Object-Store-Modells und die vier API-Operationen.
45. Warum ist LIST begrifflich besonders wichtig?
46. Durability vs. Verfügbarkeit: Unterschied in je einem Satz.
47. Vergleiche DFS und Object Store entlang Modell, Zugriff, Stärke, Grenze.
48. Was ist ein Data Lake als Architekturprinzip? Was ist ein Lakehouse — und was ist es *nicht*?

▼ **Block E + F**

49. Nenne die vier Hadoop-Kernbausteine mit ihrer jeweiligen Ebene.
50. Warum ist „YARN verwaltet Daten“ falsch?
51. Was ist der Metastore — und was ist er nicht?
52. Wie sitzt HBase auf HDFS? Beschreibe die drei Ebenen.
53. Erkläre die Kette Fachfrage → RowKey → Zugriffsweg → Passung an einem Sensorbeispiel.
54. Warum ist HBase keine verteilte relationale Datenbank?
55. Erkläre Map, Shuffle, Reduce mit je einer didaktischen Aussage.
56. Warum ist der Shuffle der zentrale Kostenpunkt?
57. Was ist ein DAG, und was unterscheidet ihn wirklich von MapReduce?
58. Transformation vs. Action vs. Lazy Evaluation.
59. Warum ist „Streaming = Echtzeit“ falsch?
60. Nenne die vier Ebenen der End-to-End-Architektur mit Leitfrage.
61. Symptom-Diagnose: Ordne „langsame Berichte“, „langsamer Zugriff“, „Daten nicht nutzbar“ je einer Ebene zu.
62. Nenne die drei typischen Fehldiagnosen.
63. Nenne die vier Vergleichskriterien aus UE24 und die vier typischen Fehlentscheidungen.
64. Erkläre das Antwortgerüst in vier Schritten.
65. Nenne die drei häufigsten Fehlzuordnungen im Gesamtmodell.